

UNIVERSIDAD ESTATAL PENÍNSULA DE SANTA ELENA

FACULTAD DE SISTEMAS Y TELECOMUNICACIONES

CARRERA DE INGENIERÍA EN SOFTWARE

ASIGNATURA:

SISTEMAS DISTRIBUIDOS Y CLOUD COMPUTING

INFORME TÉCNICO DE AUDITORÍA:

MATRIZ PRIORIZADA DE HALLAZGOS CRÍTICOS Y ALTOS

Arena de Auditoría Técnica · Fase 1 (Red Team vs Blue Team)

DATOS DE LA AUDITORÍA (FASE 1)

Repositorio auditado:	`red-social-distribuida`
Rama / SHA:	`main` (Commit verificado en entorno local)
Stack tecnológico:	Quarkus 3.33 LTS (Java 21), Neo4j 5, Redis 7, MinIO, React 19
Fecha de emisión:	Octubre 2026
Total Hallazgos Detectados:	28 hallazgos en auditoría exhaustiva
Hallazgos Priorizados:	**9 Hallazgos (1 Crítico + 8 Altos)**

EQUIPO AUDITOR (RED TEAM):

Andy Bryan Alejandro Vera
Alisson Yamel Reyes Ricardo
Gino Maximiliano Bermúdez Santos
Yandris Miguel Rivera Torres

CURSO Y PARALELO: SOFTWARE 7/1 · **DOCENTE:** ING. ANTHONY ABRAHAN PACHAY ESPINOZA

LA LIBERTAD – ECUADOR · **Octubre 2026**

Índice

1. Propósito y Justificación Estratégica del Red Team **2**
1.1. Resumen de Reproducibilidad (Misión 1: FROM ZERO) . . . 2

2. Top 3: Los Tres Problemas Técnicamente Más Relevantes **2**

3. Tabla Consolidada de Hallazgos Prioritarios (GitHub Issues) **3**

4. Fichas Técnicas de Hallazgos (Detalle para GitHub Issues) **4**
4.1. H-10 · Clave privada RSA del JWT legible por todos (chmod 644) . . . 4
4.2. H-11 · Enumeración de usuarios por Timing Attack en Login . . . 4
4.3. H-12 · Ausencia de cabecera Content-Security-Policy (CSP) en Proxy Inverso . . . 5
4.4. H-28 · Credencial inmutable: sin cambio de contraseña ni revocación de JWT . . . 6
4.5. H-05 · Acoplamiento excesivo y orquestación monolítica en PostService . . . 7
4.6. H-18 · Flujo de registro en dos round-trips obligatorios (UX degradada) . . . 7
4.7. H-19 · Pérdida irrecuperable de borradores en PostComposer . . . 8
4.8. H-22 · Pantalla en blanco (White Screen of Death) por falta de Error Boundary y Null-Safety en comentarios . . . 8
4.9. H-24 · Ausencia de Circuit Breaker y Timeout en Driver Reactivo Neo4j . . . 9

5. Guía de Registro y Próximos Pasos en la Arena (Fase 1 → Fase 2) **10**
5.1. Instrucciones para la apertura de GitHub Issues . . . 10
5.2. Criterios de Evaluación para la Fase 2 (Derecho a Réplica del Blue Team) . . . 10

1 Propósito y Justificación Estratégica del Red Team

El propósito de la **Arena de Auditoría Técnica** es confrontar el software asignado con rigor científico, evidencia reproducible y una evaluación constructiva que impulse la resiliencia del sistema.

Durante la fase de análisis estático y dinámico exhaustivo, este equipo auditor identificó un universo total de **28 hallazgos** (catalogados en el informe integral complementario). No obstante, en estricto cumplimiento de la **Regla de Oro** de la cátedra:

> *«Una opinión no constituye un hallazgo. Una impugnación sin evidencia no constituye una defensa. La calidad de los argumentos vale más que la cantidad de errores encontrados.»*

El Red Team ha tomado la decisión deliberada de **no inundar al Blue Team** con 28 observaciones que incluyan incidencias cosméticas, inconsistencias de nomenclatura o acoplamientos menores de código. Presentar decenas de issues diluye la capacidad de defensa y el valor técnico del ejercicio en la Fase 2.

Por consiguiente, el presente documento consolida exclusivamente los **Hallazgos de Severidad Crítica y Alta**. Estos 9 problemas constituyen los riesgos de seguridad más graves, los incumplimientos arquitectónicos de mayor impacto, las fallas que provocan caída completa de la interfaz en el cliente y los puntos únicos de fallo que amenazan la operación en producción. Son los hallazgos que se registrarán formalmente como **GitHub Issues** para el debate en la Fase 2.

1.1 Resumen de Reproducibilidad (Misión 1: FROM ZERO)

- **Resultado del despliegue desde cero:** Positivo **Reproducible exitosamente.**
- El comando `docker compose up -d -build` levanta de forma determinista la totalidad del ecosistema (2 instancias Quarkus, 1 réplica Neo4j, 1 Redis, 1 MinIO, 1 Nginx y contenedor frontend en Node).
- El script `scripts/seed-demo.mjs` siembra los datos iniciales y valida automáticamente las consultas Cypher (**C1** a **C8**), confirmando la correcta inicialización del grafo social.

2 Top 3: Los Tres Problemas Técnicamente Más Relevantes

Conforme a la pauta de la Fase 1, el Red Team destaca los tres problemas con mayor repercusión sistémica:

TOP 1 · H-10: Clave Privada RSA World-Readable (chmod 644)

Severidad: Crítica **CRÍTICA** · **Misión:** M4 (Seguridad)
Por qué es el #1: En `docker-compose.yml:102`, el contenedor `keys-init` ejecuta `chmod 644 /keys/*` sobre la clave privada RSA (`privateKey.pem`). Esto permite que cualquier contenedor o proceso con lectura al volumen monte o lea la clave privada, permitiendo la **falsificación universal de tokens JWT**. Un atacante puede firmar credenciales para cualquier usuario (`subject`) y asumir privilegios administrativos completos.

TOP 2 · H-28: Credencial Inmutable y Sesiones JWT Irrevocables

Severidad: Alta **ALTA** · **Misión:** M4 (Seguridad)
Por qué es el #2: No existe en toda la API un endpoint de cambio de contraseña (`PUT /auth/password`) ni recuperación. La contraseña es fija y permanente. Además, los JWT emitidos tienen validez de 24 horas y no existe lista de revocación en Redis ni `tokenVersion` en el grafo. Si una contraseña o token se filtra, la víctima no puede cambiarla ni expulsar al atacante; el secuestro de la cuenta es definitivo.

TOP 3 · H-11: Enumeración de Usuarios por Timing Attack en Login

Severidad: Alta **ALTA** · **Misión:** M4 (Seguridad)
Por qué es el #3: En `AuthService.login()`, cuando el usuario no existe en Neo4j, el flujo retorna inmediatamente sin calcular el hash `BcryptUtil.matches`. Cuando el usuario existe pero la contraseña es errónea, se consume el costo criptográfico de bcrypt (~80–120 ms). Esta asimetría temporal medible permite a un atacante enumerar la base de usuarios completa mediante un simple análisis estadístico de latencias.

3 Tabla Consolidada de Hallazgos Prioritarios (GitHub Issues)

La siguiente matriz contiene los hallazgos que conforman el informe oficial del Red Team y que han sido preparados para su apertura como GitHub Issues:

ID	Misión	Severidad	Título / Resumen Técnico	Evidencia Clave (Archivo / Línea)
H-10	M4 Seguridad	Crítica	Clave privada RSA con permisos 644 (world-readable).	`docker-compose.yml:102` (`chmod644/keys/*`)
H-11	M4 Seguridad	Alta	Timing attack y user enumeration en login.	`AuthService.java:51-58`, `AuthRepository.java:51-66`
H-12	M4 Seguridad	Alta	Ausencia de Content-Security-Policy (CSP) en Nginx.	`frontend/nginx.conf` (directivas `add_header`)
H-28	M4 Seguridad	Alta	Credencial inmutable: sin cambio de contraseña ni revocación de JWT.	`AuthResource.java:23,36,48`, `AuthProvider.tsx:68-71`
H-05	M3 Código	Alta	Acoplamiento excesivo en `PostService` (Fat Service).	`PostService.java:21-26` (5 dependencias directas)
H-18	M5 UX	Alta	Flujo de registro en dos round-trips obligatorios.	`AuthResource.java:33-40`, `users.ts:7-22`
H-19	M5 UX	Alta	Pérdida irrecuperable de borradores en `PostComposer`.	`frontend/src/components/PostComposer.tsx`
H-22	M5 UX	Alta	Pantalla en blanco (WSOD) por falta de `ErrorBoundary` y null-safety en comentarios.	`frontend/src/components/CommentThread.tsx:31-50`
H-24	M6 Resiliencia	Alta	Ausencia de Circuit Breaker y Timeout en driver Neo4j.	`ExceptionMappers.java:27`, `application.properties`

4 Fichas Técnicas de Hallazgos (Detalle para GitHub Issues)

A continuación se presentan las fichas técnicas detalladas de cada uno de los 9 hallazgos, formuladas bajo la estructura obligatoria de la auditoría.

4.1 H-10 · Clave privada RSA del JWT legible por todos (chmod 644)

Severidad: Crítica **CRÍTICA** · **Misión:** Misión 4 (Seguridad) · **CWE:** CWE-732 / CWE-552

Descripción: El script de inicialización de certificados y claves del servicio de autenticación asigna permisos de lectura globales (`644`) sobre las claves criptográficas montadas en el volumen compartido. La clave privada RSA utilizada para la firma de tokens JWT queda expuesta a cualquier proceso o contenedor que comparta el almacenamiento.
Pasos para reproducir:

1. Ejecutar el despliegue del ecosistema con `dockercomposeup-d`.
2. Inspeccionar los permisos de los archivos generados en el contenedor o volumen:

```
docker compose exec backend-1 ls -la /keys
```

3. Observar los permisos del archivo `privateKey.pem`.

Resultado esperado: La clave privada debe poseer permisos estrictos de acceso (`chmod600` o `chmod640`), siendo accesible exclusivamente por el usuario del sistema que ejecuta el proceso de Quarkus.
Resultado obtenido: Los archivos se configuran explícitamente con permisos `rw-r--r--(644)`:

```
-rw-r--r-- 1 root root 1675 privateKey.pem
```

Evidencia: Archivo `docker-compose.yml`, líneas 98–103:

```
keys-init:
  image: alpine:3.20
  volumes:
    - app_keys:/keys
  command: |
    sh -c "
      ...
      chmod 644 /keys/*   # <-- Vulnerabilidad crítica
    "
```

Impacto: Crítico. Un atacante con acceso de solo lectura en el host, mediante un escape de contenedor o acceso a volúmenes compartidos, puede extraer `privateKey.pem` y forjar tokens JWT arbitrarios con validez total. Esto vulnera el control de acceso de todos los microservicios sin requerir interacción con la base de datos.
Recomendación: Modificar `docker-compose.yml` para aplicar `chmod600/keys/privateKey.pem` y asignar el UID/GID específico del usuario no-root que ejecuta Quarkus (`chown1001:1001/keys/privateKey.pem`).

4.2 H-11 · Enumeración de usuarios por Timing Attack en Login

Severidad: Alta **ALTA** · **Misión:** Misión 4 (Seguridad) · **CWE:** CWE-208 / CWE-204

Descripción: El endpoint de autenticación `POST/auth/login` presenta una discrepancia medible en los tiempos de respuesta dependiendo de si el nombre de usuario existe o no en la base de datos, posibilitando la enumeración automatizada de cuentas registradas.
Pasos para reproducir:

1. Enviar una solicitud de login con un usuario inexistente:

```
curl -w "%{time_total}\n" -X POST http://localhost:8080/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"usuario_fantasma","password":"Password123!"}'
```

2. Enviar la misma solicitud con un usuario registrado legítimo (p. ej., `"ana"`), pero con contraseña incorrecta.

3. Comparar los tiempos de latencia devueltos.

Resultado esperado: Ambas peticiones deben responder en un intervalo de tiempo indistinguible para evitar la fuga de información sobre la existencia de cuentas.
Resultado obtenido: El usuario inexistente retorna en ∼5–15 ms (búsqueda vacía en Neo4j). El usuario existente con contraseña errónea toma entre 80–120 ms debido a la ejecución del cálculo de `BcryptUtil.matches`.
Evidencia: En `AuthService.java:51-58`:

```
public LoginResponse login(LoginRequest request) {
    Credentials credentials = repository.findCredentials(request.username())
        .orElseThrow(InvalidCredentialsException::new); // Cortocircuito rápido
    if (!BcryptUtil.matches(request.password(), credentials.passwordHash())) {
        throw new InvalidCredentialsException();
    }
    return new LoginResponse(jwtService.sign(credentials.id()), profile);
}
```

Impacto: Alto. Facilita la creación de listas verificadas de usuarios válidos del sistema mediante herramientas de fuerza bruta o escaneo distribuido, burlando la protección del rate limit de Nginx a lo largo del tiempo.
Recomendación: Computar siempre la verificación criptográfica con un hash ficticio precargado cuando el usuario no exista:

```
String hash = repository.findCredentials(request.username())
    .map(Credentials::passwordHash)
    .orElse(DUMMY_BCRYPT_HASH);
boolean valid = BcryptUtil.matches(request.password(), hash);
```

4.3 H-12 · Ausencia de cabecera Content-Security-Policy (CSP) en Proxy Inverso

Severidad: Alta **ALTA** · **Misión:** Misión 4 (Seguridad) · **CWE:** CWE-1021 / CWE-79

Descripción: La configuración de Nginx para el servidor web del frontend carece de la cabecera HTTP `Content-Security-Policy`. En un sistema que almacena tokens JWT en `localStorage`, la falta de CSP expone las credenciales de sesión ante cualquier vulnerabilidad XSS en comentarios, descripciones o metadatos de usuario.
Pasos para reproducir:

1. Inspeccionar las cabeceras HTTP de respuesta entregadas por Nginx:

```
curl -I http://localhost:80/
```

2. Verificar la ausencia de la cabecera `Content-Security-Policy`.

Resultado esperado: El servidor web debe emitir una política CSP restrictiva que limite la ejecución de scripts al propio origen (`'self'`).
Resultado obtenido: Nginx únicamente incluye `X-Content-Type-Options`, `X-Frame-Options` y `Referrer-Policy`:

```
HTTP/1.1 200 OK
Server: nginx/1.27.0
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: strict-origin-when-cross-origin
```

Evidencia: Archivo `frontend/nginx.conf`, líneas 12–16. No existe ninguna directiva `add_headerContent-Security-Policy`.
Impacto: Alto. Si un atacante inyecta un payload en el texto de un comentario o biografía, el script se ejecutará sin restricción del navegador, pudiendo acceder a `localStorage.getItem('nodouni.session')` y exfiltrar el token JWT de la víctima.
Recomendación: Agregar en `frontend/nginx.conf`:

```
add_header Content-Security-Policy "default-src 'self';
  script-src 'self'; style-src 'self' 'unsafe-inline';
  img-src 'self' data: blob:;
  connect-src 'self' wss: https://*.push.services.mozilla.com;" always;
```

4.4 H-28 · Credencial inmutable: sin cambio de contraseña ni revocación de JWT

Severidad: Alta **ALTA** · **Misión:** Misión 4 (Seguridad) · **CWE:** CWE-620 / CWE-613

Descripción: El sistema no proporciona ningún endpoint ni mecanismo para el cambio o recuperación de contraseña. Sumado a esto, los tokens JWT poseen una validez fija de 24 horas (`lifespan=86400`) y carecen de un contador de versión (`tokenVersion`) o lista negra de revocación en Redis.
Pasos para reproducir:

1. Inspeccionar las rutas expuestas en `AuthResource.java`. Solo existen `@POST/registro` y `@POST/login`.
2. Iniciar sesión y capturar el JWT emitido.
3. En el frontend, hacer clic en «Cerrar sesión».
4. Enviar una petición a un endpoint privado con el JWT previo mediante `curl`:

```
curl -H "Authorization: Bearer <TOKEN_CAPTURADO>" http://localhost:8080/api/usuarios/yo
```

5. La petición responde con `200OK` a pesar de que el usuario cerró sesión en su cliente.

Resultado esperado: Debe existir un flujo de cambio de contraseña que invalide inmediatamente todos los tokens emitidos con anterioridad y permita expulsar sesiones no autorizadas.
Resultado obtenido: El cierre de sesión es una operación exclusivamente cosmética del cliente (`setSession(null)` en `AuthProvider.tsx`). El backend no tiene forma de revocar el acceso y el usuario no puede cambiar su clave.
Evidencia:

- `backend/src/main/java/com/redsocial/auth/AuthResource.java:23,36,48`: Ningún endpoint de contraseña.
- `backend/src/main/resources/application.properties:23`: `smallrye.jwt.new-token.lifespan=86400`.
- `frontend/src/context/AuthProvider.tsx:68-71`: `logout()` solo purga el almacenamiento local.

Impacto: Alto. Secuestro definitivo de cuentas. Si la credencial o el token de un usuario se filtra, la víctima no tiene mecanismo para expulsar al intruso ni para restablecer su seguridad.
Recomendación: Implementar un atributo `tokenVersion` en el nodo `(:Usuario)` de Neo4j e incluirlo como claim en el JWT. Implementar `PUT/api/auth/password` que incremente dicho contador, invalidando en tiempo real las credenciales en ambas réplicas.

4.5 H-05 · Acoplamiento excesivo y orquestación monolítica en PostService

Severidad: Alta **ALTA** · **Misión:** Misión 3 (Calidad de Código) · **CWE:** Arquitectura / God Service

Descripción: La clase `PostService` concentra 5 colaboradores inyectados por constructor (`PostRepository`, `MediaStorage`, `ImageValidator`, `Event<PostCreated>`, `PostAssembler`). Su método `create()` coordina validaciones de texto, procesamiento de imágenes, subida a MinIO/S3, escritura en grafo Neo4j, publicación asíncrona de eventos CDI y ensamblado de respuesta, abarcando cuatro bloques `try/catch` de compensación manual.
Pasos para reproducir: Inspeccionar el constructor y el método `create()` en `backend/src/main/java/com/redsocial/posts/PostService.java:21-26,38-83`.
Resultado esperado: El servicio debe adherirse al principio de responsabilidad única (SRP), delegando la orquestación a colaboradores especializados.
Resultado obtenido: El método `create()` actúa como un monolito procedimental de 46 líneas altamente acoplado.
Evidencia: Líneas 21–26 de `PostService.java`:

```
public PostService(PostRepository repository, MediaStorage storage,
                   ImageValidator images, Event<PostCreated> events,
                   PostAssembler assembler) { ... }
```

Impacto: Alto. Alto riesgo de regresión en cada sprint. Modificaciones en el almacenamiento de imágenes o en el esquema de eventos obligan a editar la misma clase crítica de creación de posts.
Recomendación: Descomponer `PostService` en clases especializadas (`PostCreator`, `PostImageAttacher`, `PostPersister`) con transaccionalidad aislada.

4.6 H-18 · Flujo de registro en dos round-trips obligatorios (UX degradada)

Severidad: Alta **ALTA** · **Misión:** Misión 5 (Experiencia de Usuario) · **CWE:** Eficiencia de Protocolo / UX

Descripción: El endpoint `POST/auth/registro` retorna únicamente el objeto de perfil del usuario (`Profile`) sin emitir un token JWT. Esto obliga al cliente web a realizar una segunda petición inmediata a `POST/auth/login` reenviando las credenciales para poder iniciar la sesión.
Pasos para reproducir:

1. Completar el formulario de registro en `/registro`.
2. Abrir la pestaña de red de las herramientas de desarrollador del navegador.
3. Enviar el formulario y verificar las llamadas HTTP salientes.

Resultado esperado: El registro exitoso debe retornar directamente la credencial de autenticación (`LoginResponse` con JWT) para acceder de inmediato al sistema.
Resultado obtenido: Se ejecutan secuencialmente dos peticiones HTTP: `POST/registro` seguido de `POST/login`. Ambas consumen cuotas del limitador de tasa de Nginx (`20r/m`).
Evidencia: En `frontend/src/services/users.ts:7-22`:

```
await api.post<Profile>("/auth/registro", data);
// Segunda llamada obligatoria para obtener el token:
const login = await api.post<LoginResponse>("/auth/login", {
  username: data.username,
  password: data.password
});
```

Impacto: Alto. Duplica la latencia percibida por el usuario en el onboarding, expone la contraseña a un segundo tránsito innecesario en la red y agota prematuramente el límite de peticiones por minuto ante pruebas o registros legítimos simultáneos.
Recomendación: Hacer que `AuthService.register()` genere y retorne inmediatamente el `LoginResponse` con el token emitido, eliminando la segunda llamada desde el frontend.

4.7 H-19 · Pérdida irrecuperable de borradores en PostComposer

Severidad: Alta **ALTA** · **Misión:** Misión 5 (Experiencia de Usuario) · **CWE:** Resiliencia de Cliente / UX

Descripción: El componente de creación de publicaciones (`PostComposer`) almacena el texto del mensaje y el archivo de imagen adjunto únicamente en el estado local de React (`useState`). Si el usuario recarga la pestaña, sufre una desconexión o navega momentáneamente a otra ruta, todo el contenido redactado se descarta de forma irrecuperable sin advertencia alguna.
Pasos para reproducir:

1. Navegar a `/feed` y redactar una publicación extensa en el compositor.
2. Seleccionar una imagen adjunta.
3. Presionar la tecla `F5` (recargar página) o hacer clic en el enlace «Explorar» y regresar al «Feed».

Resultado esperado: El contenido redactado debe conservarse en el almacenamiento local (`sessionStorage`) y restaurarse automáticamente al volver al componente.
Resultado obtenido: El estado se reinicia a vacío (`""`). El texto y la imagen cargada desaparecen por completo.
Evidencia:
El componente `frontend/src/components/PostComposer.tsx` carece de ganchos `useEffect` asociados a `sessionStorage`.
Impacto: Alto. Severa degradación de la experiencia de usuario y frustración directa, particularmente en usuarios móviles o en conexiones intermitentes.
Recomendación: Implementar un guardado automático con *debounce* de 500 ms en `sessionStorage` para el texto del borrador y restaurarlo al montar el componente.

4.8 H-22 · Pantalla en blanco (White Screen of Death) por falta de Error Boundary y Null-Safety en comentarios

Severidad: Alta **ALTA** · **Misión:** Misión 5 (Experiencia de Usuario / Robustez de UI) · **CWE:** CWE-754 / CWE-248

Descripción: El componente `CommentThread` accede directamente a las propiedades del autor (`comment.autor.id`, `comment.autor.nombre`, `comment.autor.username`) asumiendo que el objeto `autor` siempre está definido. Si en la base de datos de grafos un usuario fue dado de baja o existen comentarios huérfanos, el backend retorna registros con autor nulo, desatando una excepción fatal no controlada de JavaScript (`TypeError: Cannotreadpropertiesofundefined`). Al carecer la aplicación React 19 de componentes `<ErrorBoundary>` en sus rutas y vistas de publicación, el ciclo de vida de React desmonta la totalidad del árbol de componentes del DOM, colapsando la interfaz a una pantalla completamente en blanco («White Screen of Death» / WSOD) e imposibilitando cualquier recuperación sin reiniciar o forzar recarga.
Pasos para reproducir:

1. Desconectar o eliminar un nodo de usuario en Neo4j que posea comentarios asociados en un post, o simular una respuesta de la API en `GET/posts/:id/comentarios` con un nodo donde `autor` sea `null`.
2. Navegar desde la interfaz web a la vista de detalle de dicha publicación (`/posts/:id`).
3. Abrir la consola de herramientas de desarrollador (F12).
4. Constatar el error fatal `UncaughtTypeError:Cannotreadpropertiesofundefined(reading'id')` y verificar que la pantalla de la aplicación queda totalmente en blanco.

Resultado esperado: El componente debe emplear navegación segura (*optional chaining*) con valores por defecto (p. ej., «Usuario no disponible»), y las secciones dinámicas deben estar protegidas por un `<ErrorBoundary>` que muestre una tarjeta de degradación elegante sin derribar la página completa.
Resultado obtenido: Colapso destructivo total de la aplicación cliente en el navegador. La vista completa se desmonta sin advertencia ni posibilidad de interacción.
Evidencia: En `frontend/src/components/CommentThread.tsx`, líneas 31–50:

```
const profilePath = "/usuarios/" + encodeURIComponent(comment.autor.id);
const firstName = comment.autor.nombre.split(" ")[0];
...
<Avatar name={comment.autor.nombre} />
...
<span className="muted break-all">@{comment.autor.username}</span>
```

Adicionalmente, una búsqueda global en `frontend/src/` confirma la ausencia absoluta de componentes `ErrorBoundary`.
Impacto: Alto. Quiebra la disponibilidad del cliente frontend. Un solo comentario con inconsistencia de autor inhabilita de forma permanente la visualización de la publicación para cualquier visitante de la red social.
Recomendación: 1. Envolver la jerarquía de rutas y componentes dinámicos en componentes `<ErrorBoundary>` con interfaz de recuperación y botón de reintento.
2. Aplicar encadenamiento opcional en `CommentThread.tsx`: `comment.autor?.username??copy.deletedUser`.

4.9 H-24 · Ausencia de Circuit Breaker y Timeout en Driver Reactivo Neo4j

Severidad: Alta **ALTA** · **Misión:** Misión 6 (Resiliencia) · **CWE:** CWE-400 / Resiliencia Distribuida

Descripción: Las consultas dirigidas a Neo4j mediante la sesión reactiva carecen de un límite de tiempo de espera (*timeout*) a nivel de petición y de un patrón de disyuntor (*Circuit Breaker*). Si el motor de grafos se bloquea por saturación de CPU o I/O, las solicitudes entrantes en Quarkus quedan suspendidas indefinidamente.
Pasos para reproducir:

1. Pausar el contenedor de Neo4j mediante `dockercomposepauseneo4j`.
2. Realizar múltiples peticiones concurrentes a `GET/api/feed`.
3. Observar el consumo de conexiones y el tiempo de respuesta del balanceador Nginx.

Resultado esperado: El backend debe abortar la consulta en un tiempo máximo prudente (≤ 3 s), activar un disyuntor y retornar una respuesta degradada o un error 503 controlado.
Resultado obtenido: Las peticiones se quedan colgadas esperando el socket de Neo4j hasta agotar los hilos de eventos de Netty/Quarkus, provocando una caída en cascada de ambas réplicas del backend y respuestas `504 Gateway Timeout` en Nginx.
Evidencia:
Configuración en `application.properties` y ausencia de anotaciones `@Timeout` o `@CircuitBreaker` (SmallRye Fault Tolerance) en los repositorios de Neo4j.
Impacto: Alto. Pérdida de tolerancia a fallos. La ralentización de un único servicio de almacenamiento satura y tumba la totalidad del clúster de servidores de aplicación.
Recomendación:
Integrar `quarkus-smallrye-fault-tolerance` y anotar los métodos de lectura de repositorios con `@Timeout(value=3,unit=ChronoUnit.SECONDS)` y `@CircuitBreaker(requestVolumeThreshold=10,failureRatio=0.5)`.

5 Guía de Registro y Próximos Pasos en la Arena (Fase 1 → Fase 2)

5.1 Instrucciones para la apertura de GitHub Issues

Cada uno de los 9 hallazgos detallados en la Sección 4 debe ser registrado en el repositorio auditado bajo el siguiente formato estandarizado:

- **Título del Issue:** `[RED TEAM][<ID>][<SEVERIDAD>] <Título conciso>`
- **Etiquetas sugeridas:** `bug`, `security`, `ui-ux`, `red-team-audit`, `p1-critical` / `p2-high`.
- **Cuerpo del Issue:** Copiar fielmente la ficha técnica (Descripción, Pasos de Reproducción, Resultado Esperado, Resultado Obtenido, Evidencia, Impacto y Recomendación).

5.2 Criterios de Evaluación para la Fase 2 (Derecho a Réplica del Blue Team)

Durante la Fase 2, el equipo propietario (Blue Team) deberá responder técnicamente a cada issue abierto bajo uno de los tres estados oficiales:

1. **ACEPTADO:** El Blue Team reconoce la existencia del problema y propone su plan de remediación.
2. **ACEPTADO CON OBSERVACIONES:** Reconoce el fallo técnico, pero debate con pruebas el nivel de severidad asignado o su impacto práctico.
3. **IMPUGNADO:** Rechaza formalmente el hallazgo. **Condición obligatoria:** Toda impugnación debe estar sustentada en pruebas de código, logs de ejecución o demostraciones reproducibles. Argumentaciones sin evidencia técnica no serán admitidas.

Equipo Auditor Red Team · UPSE 2026
Sistemas Distribuidos y Cloud Computing